

Un web server minimale in Java

- Creiamo un "mini" web-server in Java con queste limitazioni:
 - ▼ il server gestisce solo richieste HTTP di tipo "GET"
 - ▼ il server risponde solo a richieste di pagine html statiche e immagini jpeg
 - ▼ il server chiude la connessione al termine di ogni richiesta HTTP (quindi NON supporta la ricezione di più richieste sulla stessa connessione)
 - ▼ il server attende richieste sulla porta 1234
 - ▼ le pagine html e le immagini sono memorizzate nella cartella **c:/mioserver** del server (root del sito)
- Per provare il server occorre digitare nella barra degli indirizzi del browser la seguente URL: **http://localhost:1234/prova.html**
(il file prova.html dovrà trovarsi nella cartella **c:/mioserver**)

Esempio 6: la classe MyWebServer

```
public class MyWebServer {  
    public static void main(String[] args) throws Exception {  
        ServerSocket ss = new ServerSocket( 1234 );  
        System.out.println("Server pronto!");  
        while( true ) {  
            Socket s = ss.accept();  
            System.out.println("Connesso con "+ s.getRemoteSocketAddress());  
            MyWorker w = new MyWorker(s);  
            w.start();  
        }  
    }  
}
```

Esempio 6: la classe MyWorker

```
public class MyWorker extends Thread {  
    private static String root = "C:/mioserver";  
    private Socket s;  
    public MyWorker(Socket socket) {  
        s = socket;  
    }  
    public void run() {  
        try {  
            BufferedReader in = new BufferedReader(  
                new InputStreamReader( s.getInputStream() ) );  
            OutputStream out = s.getOutputStream();  
            String request = in.readLine();  

```

Esempio 6: la classe MyWorker

```
String response = "";
String[] tokens = request.split(" ");
if (tokens.length == 3 && tokens[0].equals("GET")) {
    File file = new File( root + tokens[1].toLowerCase() );
    if ( file.exists() ) {
        response = "HTTP/1.1 200 OK\r\n" +
            "Connection: Close\r\n" +
            "Content-Type: " + getType(tokens[1]) + "\r\n" +
            "Content-Length: " + file.length() + "\r\n\r\n";
        out.write( response.getBytes() );
        FileInputStream fileIn = new FileInputStream(file);
        int b;
```

Esempio 6: la classe MyWorker

```
        while((b = fileIn.read()) != -1 ) { out.write(b); }
    }
    else { // file non trovato
        response = "HTTP/1.1 404 Not Found\r\n" +
            "Connection: Close\r\n" +
            "\r\n" +
            "<html><head></head><body>PAGINA NON TROVATA!!! " +
            "</body></html>";
        out.write( response.getBytes() );
    }
}
s.close();
```

Esempio 6: la classe MyWorker

```
    } catch (IOException ex) {System.out.println(ex);}
} // fine metodo run

String getType(String filename) {
    if ( filename.endsWith(".html") )
        return "text/html";
    else if ( filename.endsWith(".jpg") )
        return "image/jpeg";
    return null;
}

} // fine classe MyWorker
```